

Binarios SUID

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El bit SUID hace que un ejecutable corra siempre como su propietario, sin importar quien lo lance. Si el dueño es root y el binario permite ejecutar comandos, leer/escribir ficheros o lanzar otra shell, puedes abusar de el para actuar como root. Es una aplicacion directa del privesc-modelo.

SUID existe para casos legítimos (passwd necesita tocar /etc/shadow). El problema es cuando un binario con SUID tiene una funcion que permite salir a una shell o leer ficheros arbitrarios: entonces esa capacidad se ejecuta como root. GTFOBins cataloga exactamente que binarios son abusables y como.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la fase de privesc Linux, justo despues de mirar sudo-abuse. Busca binarios SUID que NO sean del sistema base o cuya version sea conocida por ser abusable.

Cómo identificarla

- find de SUID devuelve binarios raros (nmap, find, cp, python, vim, tar) fuera de los tipicos.
- Un binario a medida del reto con el bit SUID puesto.
- El binario aparece en GTFOBins con la etiqueta SUID.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Busca el binario candidato en GTFOBins (seccion SUID) y ejecuta su linea de abuso exacta.

Evidencia esperada: Un prompt nuevo con # en vez de \$, y euid=0 en id.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
find / -perm -4000 -type f 2>/dev/null
```

Objetivo y contexto: Lista todos los binarios con bit SUID. -perm -4000 filtra justo ese bit; 2>/dev/null oculta el ruido de permiso denegado.

Resultado esperado: Rutas de binarios SUID. Ignora los típicos del sistema; cualquier cosa rara o a medida -> GTF0Bins. Anota también la versión si es un binario conocido.

Anatomía de los payloads

La primera prueba es `find / -perm -4000 -type f 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un prompt nuevo con # en vez de \$, y `euid=0` en `id`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lista los binarios SUID del sistema.
- Descarta los normales (`passwd`, `sudo`, `mount`, `ping`) y quedate con lo inusual.
- Busca cada candidato en GTF0Bins filtrando por 'SUID': te da la línea exacta.
- Ejecuta el abuso y confirma con `id` que tienes `euid=0`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Todos los SUID listados son binarios estándar del sistema sin función de escape conocida en GTF0Bins.
- El binario SUID es de tu propio usuario (dueño = tu uid, no root): no gana privilegio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un binario SUID se ejecuta con los privilegios de su dueño (a menudo root), no con los tuyos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (suid)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.